

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет прикладной математики и информатики

Кафедра вычислительной математики и программирования

Компьютерная графика
Отчёт по лабораторным работам

Некрасов К.Д.
Группа: 8О-310Б
Руководитель:
Бахарев В.Д.

Москва, 2024

Цель

В данной лабораторной работе вам предстоит научиться работать с графическим API для отрисовки 2D-примитивов, освоить основные 2D-трансформации (перемещение, масштабирование, поворот) и изучить алгоритмы построения 2D-кривых.

Требования

Вы должны использовать C++ (OpenGL + SFML) или C# (OpenTK).

Программа должна работать в реальном времени, обновляя изображение в цикле.

Визуальный результат необходимо продемонстрировать на экране с возможностью управления через интерфейс.

Задание: Вариант 3

Создайте многоугольник с произвольным числом сторон (например, пятиугольник).

Реализуйте его перемещение по экрану с помощью клавиатуры.

Добавьте возможность масштабирования и вращения многоугольника вокруг его центра.

Дополнительно: Реализуйте замкнутую траекторию движения многоугольника по экрану.

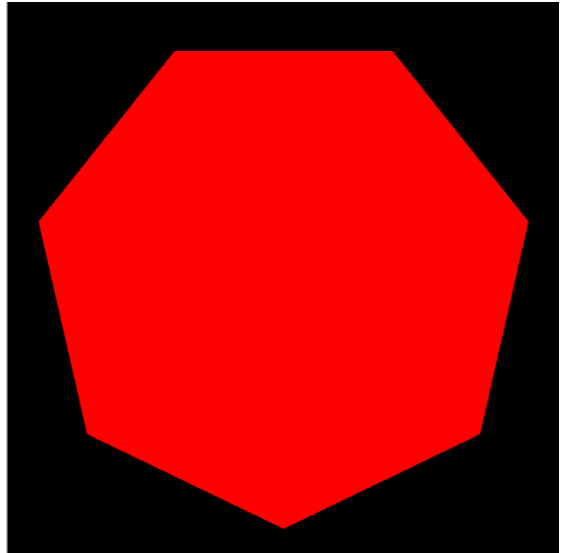
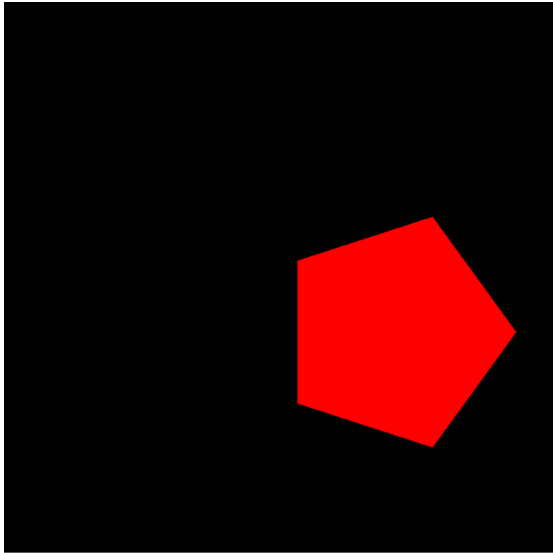
Практическая часть

Для отрисовки многоугольника создана функция `drawPolygon`, которая вычисляет координаты вершин многоугольника с учётом текущего угла поворота и масштаба, а также учитывает произвольное количество граней посредством вычисления центрального угла между соседними вершинами.

Перемещение многоугольника по экрану, масштабирование и вращение осуществляются при помощи глобальных переменных, которые изменяются посредством обработки ввода с клавиатуры `processInput`.

Замкнутая траектория движения активируется при нажатии пробела, в ней изменяют x и y координаты в зависимости от радиуса окружности, по которой многоугольник вращается.

Скриншоты



Цель

В этой лабораторной работе вы познакомитесь с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научитесь работать с матрицами перспективы и ортографической проекции.

Требования

Вы должны использовать OpenGL + SFML или OpenTK + C#.

Программа должна работать в реальном времени, с возможностью динамической смены проекции и трансформаций объектов.

Все объекты должны корректно отрисовываться с учетом проекции и иметь возможность взаимодействия с пользователем.

Задание: Вариант 6

Постройте 3D-куб.

Для каждой грани куба рассчитайте и визуализируйте нормали.

Используйте перспективную проекцию для отображения куба на экране.

Дополнительно: Реализуйте вращение куба вокруг осей, так чтобы нормали менялись в зависимости от поворота.

Практика

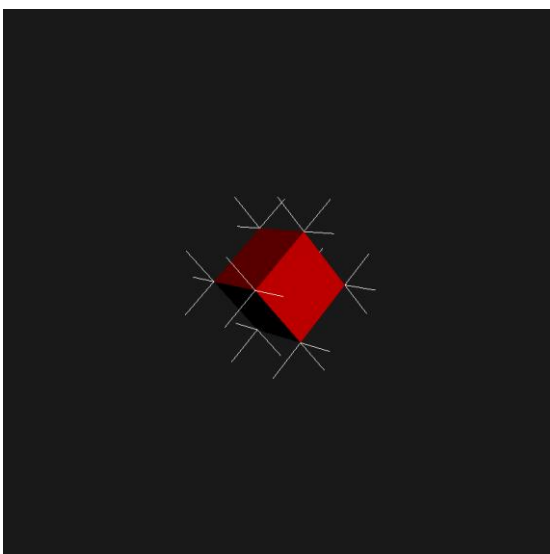
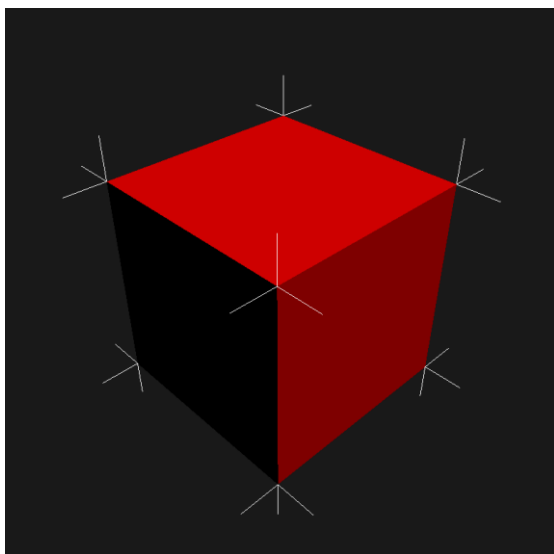
Написан шейдер для вычисления позиции вершин и их нормалей.

Для отрисовки нормалей написан отдельный шейдер, который визуализирует нормали как две соединенные точки, одна из которых находится на вершине куба.

Вращение куба осуществляется обработкой ввода и изменение глобальных переменных. Вместе с кубом, осуществляется вращение нормалей, которые вычисляются динамически.

Также был написан фрагментный шейдер для диффузного освещения граней куба.

Скриншоты



Цель

В этой лабораторной работе вы научитесь работать с камерой в 3D-пространстве, управлять её положением и направлением, а также освоите базовые трансформации (перемещение, поворот и масштабирование) объектов в 3D.

Требования

Вы должны использовать OpenGL + SFML или OpenTK + C#.

Программа должна работать в реальном времени, с возможностью динамической смены положения камеры и трансформаций объектов. Управление камеры должно быть назначено на клавиатуру или мышь.

Все объекты должны корректно отрисовываться с учетом положения камеры и примененных трансформаций.

Задание: Вариант 8

Постройте один 3D-объект (например, сферу).

Реализуйте базовые трансформации (перемещение, масштабирование, поворот) с использованием матричных операций.

Обеспечьте возможность изменения порядка применения матриц (например, сначала поворот, затем перемещение или наоборот).

Дополнительно: Добавьте визуализацию матриц трансформаций на экране в виде текстовой информации.

Практика

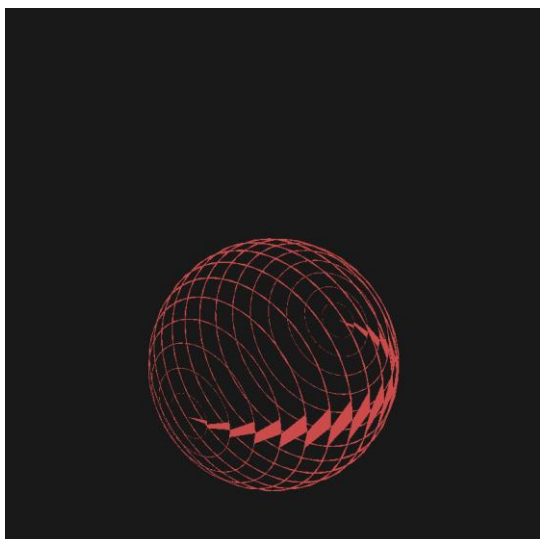
Написан шейдер для вычисления позиций вершин сферы на экране.

Массив вершин сферы при использовании сферических координат для вычисления формируется по радиусу сферы, количеству секторов и количеству стеков.

Порядок применения матриц трансформации контролируется изменением переменной `transformationOrder` в функции обработки ввода.

Вывод матрицы трансформации осуществляется в консоль

Скриншоты



```
0.798636 0 -0.601815 -2.97515e-08  
-0.215671 0.93358 -0.286205 -0.700185  
0.561843 0.358368 0.745591 -0.268776  
0 0 0 1  
0.798636 0 -0.601815 -2.97515e-08  
-0.215671 0.93358 -0.286205 -0.700185  
0.561843 0.358368 0.745591 -0.268776  
0 0 0 1
```

Цель

В этой лабораторной работе вы научитесь работать с освещением в 3D-пространстве, используя различные типы источников света, и освоите основы написания шейдеров. Вы реализуете освещение объектов в сцене с использованием простейших моделей освещения и настроите эффекты при помощи вершинных и фрагментных шейдеров.

Требования

Вы должны использовать OpenGL + SFML или OpenTK + C#.

Программа должна корректно отображать освещение с учетом типов источников света, используя написанные вами шейдеры.

В каждом варианте задания должны быть задействованы как минимум один тип освещения (например, направленное освещение, точечный источник света или прожектор).

Задание: Вариант 5

Постройте куб в 3D-пространстве.

Реализуйте плоское затенение (Flat Shading), при котором каждый полигон объекта имеет один цвет, основанный на нормали грани.

Используйте только вершинный шейдер для выполнения освещения.

Дополнительно: Реализуйте возможность переключения между плоским и гладким затенением (Gouraud Shading).

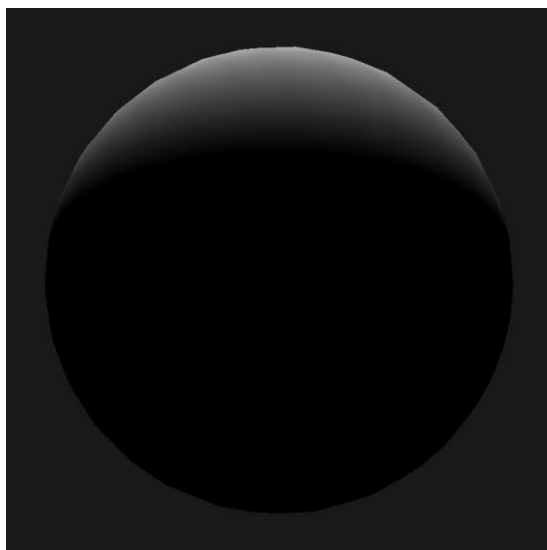
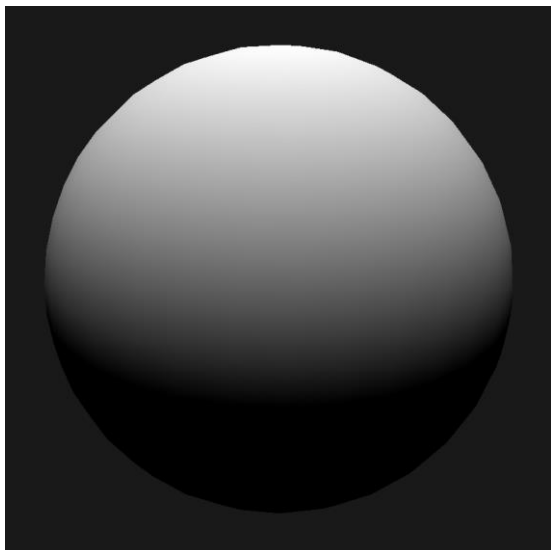
Практика

Вместо куба была построена сфера из-за её большей полигональности.

Генерируется массив вершин сферы с нормальными векторами по широте и долготе единичной сферы (сферы радиуса 1).

Шейдеры используют основное освещение направленным источником света, интенсивность света определяется как максимум скалярного произведения нормали и направления света и нуля.

Скриншоты



Цель

В этой лабораторной работе вы научитесь работать с техникой трассировки лучей для создания реалистичной 3D-графики. Вы реализуете алгоритм Ray Tracing, который позволяет рассчитывать физически корректные отражения, преломления, тени и свет в сцене. Лабораторная работа подводит к пониманию основ рендеринга, работающего с лучами света, а также к созданию реалистичных сцен.

Требования

Реализуйте алгоритм трассировки лучей для отрисовки простой сцены, используя минимальный набор примитивов (сферы, плоскости и т.д.).

Реализуйте базовые эффекты: отражения, тени и освещение.

Трассировка должна быть реализована как на CPU, так и с возможной оптимизацией на GPU (опционально).

Программа должна корректно отображать сцены в зависимости от выбранного задания.

Задание: Вариант 4

Постройте сцену с тремя сферами и одной плоскостью.

Реализуйте трассировку лучей, чтобы каждый объект мог как отражать, так и преломлять свет.

Включите обработку множественных отражений и преломлений, где лучи света могут многократно взаимодействовать с объектами сцены.

Дополнительно: Реализуйте контроль глубины трассировки для оптимизации производительности.

Практика

Для объектов на сцене определены свои структуры, для материалов объектов определены необходимые свойства (цвет, коэффициенты отражения и преломление, показатель преломления).

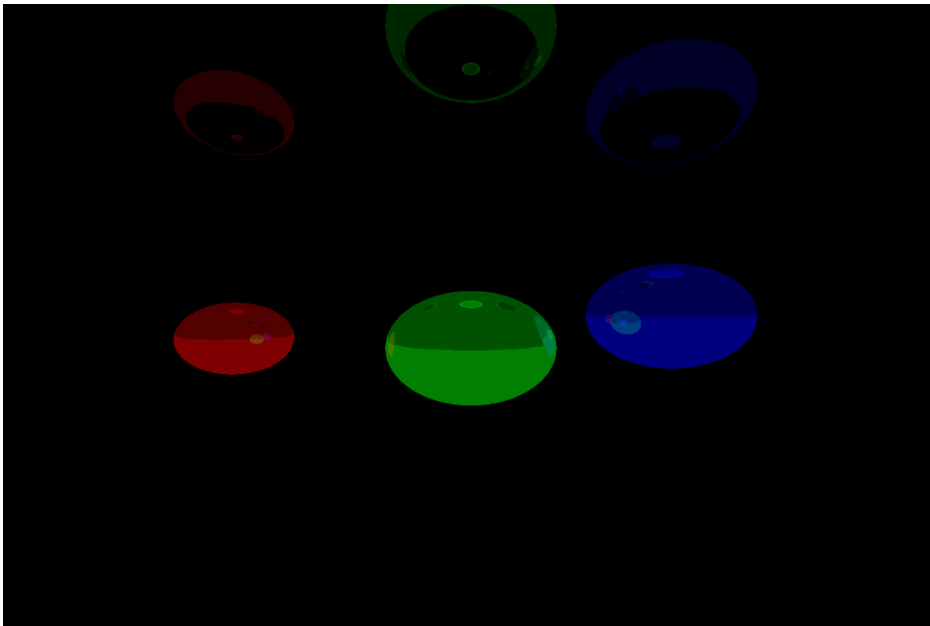
Функции пересечения с объектами определяют, пересёкся ли луч со сферой или плоскостью, а также расстояние до точки пересечения и нормаль в точке пересечения.

Функция трассировки луча является рекурсивной, и рекурсия завершается когда в процессе декрементации параметр глубины трассировки становится равен 0. Внутри функции определяется цвет в точке пересечения с учётом отражений и преломлений. Цвет по умолчанию (если пересечений не было) – чёрный.

Итоговый результат сохраняется в png изображении.

Изображения рендеров

Плоскость со 100% отражением над сферами



Плоскость отделяет синюю сферу

